

软件实现课后作业

说明：相较于之前提交的软件实现，本项目进行了大改：增加了Grafana等运维工具，将数据库从MySQL转为PostgreSQL，集成了ElasticSearch索引搜索，配置了基于Langchain4j框架+火山引擎的智能AI Agent用户助手。本总结分别从危险语言特性、项目代码风格、圈复杂度三个角度分析。

1. 危险语言特性：Java 反射/动态调用

维度	说明
危险点	跳过编译器检查、破坏封装、异常只在运行时暴露、性能与安全不可控。凡依赖运行时元数据的特性都会弱化静态类型体系，从而增加维护成本。
理论依据	软件工程理论强调“能在编译期发现的错误越多越好”；反射恰恰削弱了这一点，因此只应在框架/基础设施层使用。
团队策略	- 框架层（Spring / MyBatis）允许反射。 - 业务层禁止自建反射调用；如确需动态行为，必须封装在独立工具类、补充日志与单测并经过 Code Review。 - 优先使用接口、多态或策略模式实现扩展，避免“灵活”驱动的失控。

结论：不完全禁用，但要在编码规范中 **默认禁止业务层反射**，除非提供充分理由和保护措施。

2. 代码风格与可维护性检查

2.1 AIAgentService (njumarket-service-commodity/vector/AIAgentService.java)

项	现状	优化建议
缩进/	统一 4 空格，链式调用对齐良好	保持不变

项	现状	优化建议
格式		
命名	<code>shoppingAssistant</code> 、 <code>conversationVectorService</code> 等语义清晰。但 <code>context</code> 、 <code>memoryId</code> 在 100+ 行方法里语义略弱	提升可读性，考虑 <code>sessionContext</code> 、 <code>activeMemoryId</code> 等更具体命名
职责划分	<code>chat()</code> 方法包含：上下文准备、历史查询、调用 LLM、落库等多职责，长度 > 150 行	拆分成 <code>prepareContext()</code> 、 <code>invokeAssistant()</code> 、 <code>persistHistory()</code> 等私有方法；缩短方法、降低耦合
代码重复	<code>chat()</code> 与 <code>chatStream()</code> 重复逻辑（画像、历史检索、ContextMap 维护）	提取公共模板或抽象辅助类；流式与非流式只处理差异部分
状态共享	<code>SearchCommoditiesTool</code> 的 <code>lastSearchResults</code> 是共享可变成员	优化：让 Tool 返回搜索结果，外部方法接收并存入上下文，避免并发覆盖

2.2 CommodityQueryServiceImpl (.../service/impl/CommodityQueryServiceImpl.java)

项	现状	优化建议
函数长度	<code>searchCommodities(...)</code> 200+ 行，集成 ES 回退、分页、过滤、DTO 组装等步骤	拆分： <code>trySearchCommoditiesWithElastic</code> 、 <code>fallbackSearch</code> 、 <code>buildPageResult</code> 已初步拆出，但内部仍可进一步分段（如“价格过滤”“DTO 转换”等）
命名	<code>elasticResult</code> 、 <code>filteredCommodities</code> 清楚，但	适度添加注释或改名为 <code>validatedPage</code>

项	现状	优化建议
	safePage、safeSize 等细节建议 写注释说明“校验后的值”	
缩进	stream().filter().collect() 等表达式较长，建议分行或提炼局部变量	可提升可读性
并发/缓存	方法内已批量查询并缓存卖家信息 (防止 N+1)，符合规范	继续保持

整体来看，项目大部分文件遵循统一风格，但长方法/共享状态仍是主要技术债，需要逐步重构。

3. 圈复杂度分析

AIAgentService.chat(...)

指标	数值 / 说明
圈复杂度	9（基础 1 + 8 个决策点：参数校验、if (StringUtils.hasText(...))、try/catch、if (memoryMessages.size() >= ...)等）
问题	虽未超过 10，但方法行数长、职责多，理解/维护成本高。
风险	- 阅读困难：新人需要追踪多个职责。 - 测试困难：要覆盖所有分支需大量用例。 - 重复逻辑：与 chatStream() 共享步骤却无法重用。
降低复杂度方案	- 拆分方法：prepareContext、shouldSummarize，persistConversation. - 引入 ChatSessionContext DTO，集中存储 userProfileContext、conversationSummary 等。 - 使用模板/回调封装“执行 LLM + 获取搜索结果”步骤。 - 语句减少深层嵌套。

结语

- 危险特性（反射）已有清晰的团队约束：框架层允许，业务层默认禁止。
- 风格与复杂度检查表明，项目在命名/缩进方面整体规范，但仍需通过“拆分长方法、消除共享状态”来减轻维护压力。
- 圈复杂度的定量分析（以 `AIAgentService` 为例）证明了继续重构的必要性。